

บทที่ 3

วิธีดำเนินการทำงาน

3.1 บทนำ

เนื่องจากปฏิญานิพนธ์ทั้งหมดประกอบด้วยส่วนการทำงานหลัก 3 ส่วนได้แก่ ส่วนโปรแกรม ส่วนวงจรอิเล็กทรอนิกส์ และส่วนงานทางกล เห็นได้ชัดว่าต้องใช้ความรู้ในการทำงานหลายด้าน ดังนั้น การปรับตัวในการทำงานที่หลากหลายจึงเป็นสิ่งจำเป็น นอกเหนือจากการแบ่งงาน และแบ่งเวลาที่เหมาะสมแล้ว ผู้จัดทำได้แบ่งขั้นตอนการทำงานที่เหมาะสม ดังต่อไปนี้

3.2 การศึกษาข้อมูล

3.2.1 ศึกษาการทำงานของ Linux kernel

Linux kernel เป็น โปรแกรมหลักของระบบที่ทำหน้าที่ดูแลการใช้งานโปรแกรมและฮาร์ดแวร์ (Hardware) ต่างๆ ของผู้ใช้ให้เป็นระเบียบและถูกต้อง โครงสร้างภายในของ Linux Kernel ประกอบด้วย ส่วนต่างๆ ดังนี้

- **Process management**

ทำหน้าที่สร้างโปรเซส (Process) ที่เข้ามาใหม่และดูแลโปรเซสที่กำลังทำงานอยู่และทำลายโปรเซสที่ไม่ต้องการในระบบ

- **Memory management**

ทำหน้าที่ดูแลและจัดการการใช้งานหน่วยความจำ (Memory) ของโปรเซสทั้งหมดไม่ให้เกิดการแย่งใช้งานหรือเลื่อมล้ำพื้นที่ทำงาน

- **File system**

ลินุกซ์จะมองทุกอย่างเป็นไฟล์ (File) และสามารถใช้งานกับระบบไฟล์ได้หลายระบบบนลินุกซ์ ดังนั้นจึงต้องมีส่วนที่ดูแลจัดการระบบไฟล์ดังกล่าว

- **Device control**

เป็นส่วนที่ทำหน้าที่ดูแลการใช้งานอุปกรณ์ภายนอกทั้งหมด ตั้งแต่ฟลอปปีดิสก์ (Floppy Disk) ไปจนถึงปริ้นเตอร์ (Printer) เพราะลินุกซ์จะไม่ยอมให้ผู้ใช้สามารถใช้งานอุปกรณ์ภายนอกได้โดยตรงเพื่อป้องกันการยึดครองอุปกรณ์ ดังนั้นเคอร์เนลจึงมีระบบที่คอยจัดการการทำงานดังกล่าว

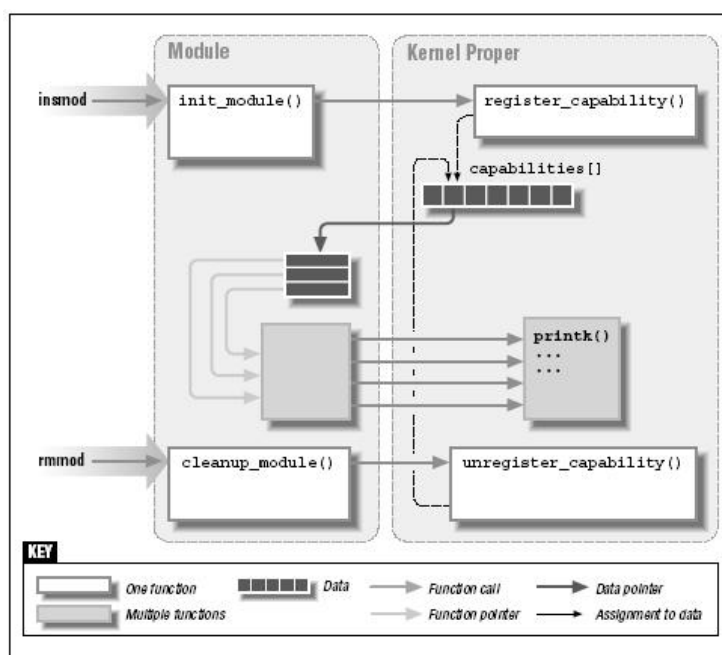
- **Networking**

เป็นอีกส่วนหนึ่งที่ระบบปฏิบัติการต้องดูแล เพราะข้อมูลที่เข้าออกผ่านระบบเน็ตเวิร์ก (Network) จะเป็นแบบอะซิงโครนัส (Asynchronous) ดังนั้นจึงต้องมีการติดตามดูแลการทำงานของแพ็กเกจ (Packet) ตลอดเวลา

หมายเหตุ สามารถหารายละเอียดของเคอร์เนลเพิ่มเติมได้จาก www.kernel.org

3.2.2 ศึกษาวิธีการเขียน Linux Device driver

เหตุผลที่ต้องใช้ Linux Device driver นั้นผู้ใช้ควรทราบก่อนว่าลินุกซ์จะแบ่งพื้นที่การทำงานออกเป็น 2 ส่วน คือ user space และ kernel space ซึ่งผู้ที่ใช้งานอยู่บน user space จะไม่สามารถเข้าใช้งานโปรแกรมบน kernel space ได้โดยตรง การใช้งานจะต้องกระทำผ่าน system call เท่านั้น ดังนั้นการสร้างแอปพลิเคชัน (Application) ที่สามารถแบ่งปันทรัพยากรของระบบได้นั้นจะต้องสร้างขึ้นภายใน kernel space โดยการสร้างโมดูล (Module) ที่ทำงานตามที่ต้องการจากนั้นนำโมดูลที่ได้ไปวางไว้ในระดับ kernel space เพื่อผนวกเข้ากับเคอร์เนลโดยการใช้คำสั่ง insmod เพียงเท่านี้โปรแกรมที่สร้างขึ้นจะสามารถรองรับการทำงานให้กับยูสเซอร์ (User) ทุกคนได้ ซึ่งการนำโมดูลเข้าและออกนั้นสามารถแสดงได้ดังรูป

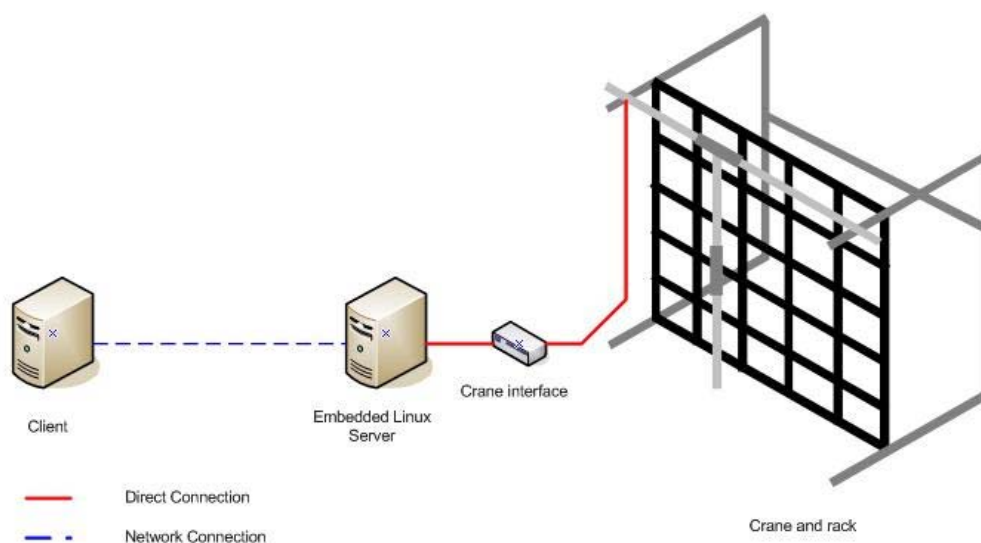


รูปที่ 3-1 แสดงการทำงานของคำสั่ง insmod และ rmmod

ในรูปที่ 3-1 แสดงให้เห็นว่าเมื่อมีการใช้คำสั่ง insmod จะมีการรันฟังก์ชัน (run function) init_module() ของโมดูลเพื่อทำงานบางอย่าง เช่น ตรวจสอบว่ามีทรัพยากรว่างหรือไม่ หรือแสดงสถานะของโมดูลก่อนถูกใช้งาน เป็นต้น

3.3 การออกแบบระบบ

3.3.1 โครงสร้างและการทำงานของระบบโดยรวม



รูปที่ 3-2 แสดงโครงสร้างโดยรวมของระบบ

- **Client**

เป็นส่วนที่ผู้สเซอร์สามารถสั่งงานได้จากระยะไกลมายังเซิร์ฟเวอร์ซึ่งสามารถมีได้มากกว่า 1 เครื่อง แต่ไม่สามารถสั่งงานได้พร้อมกันและใช้ได้กับเครื่องคอมพิวเตอร์ที่สามารถเปิดเว็บเบราว์เซอร์ (Web Browser) และรองรับการทำงานระบบเน็ตเวิร์ก

- **Embedded Linux Server**

เป็นส่วนควบคุมหลักของระบบทั้งหมดและในการทดลองนี้จะให้ความสำคัญกับส่วนนี้เป็นพิเศษ โดยเซิร์ฟเวอร์จะถูกออกแบบให้ทำงานบนระบบปฏิบัติการลินุกซ์และถูกปรับแต่งให้ทำงานแบบ Embedded system ซึ่งโครงสร้างภายในจะใช้เครื่องคอมพิวเตอร์ที่มีสถาปัตยกรรมตระกูล x86

- **Crane interface**

เป็นส่วนที่ติดต่อระหว่างเซิร์ฟเวอร์กับอุปกรณ์ที่ติดตั้งบนเครนทั้งหมดทำหน้าที่แปลงระดับสัญญาณระหว่างดิจิทัล (Digital) ให้เป็นระดับแรงดันที่อุปกรณ์ภายนอกต้องการ หรือทำหน้าที่แปลงระดับแรงดันที่เกิดขึ้นภายนอกให้อยู่ในระดับสัญญาณทีแอล (TTL) เพื่อป้อนให้กับพอร์ตขนาน (Parallel port) เป็นต้น

- **Crane and Rack**

เป็นส่วนที่ทำหน้าที่หลักทางกลโดยโครงสร้างหลักจะประกอบด้วยเครนและโครง (Rack) ซึ่งเครนจะเป็นส่วนที่เคลื่อนที่ได้ทั้งแนวตั้งและแนวนอน อีกส่วนคือโครงจะเป็นส่วนที่ทำหน้าที่เสมือนชั้นวางของ หรืออาจนำไปประยุกต์แทนการใช้งานด้านอื่นได้

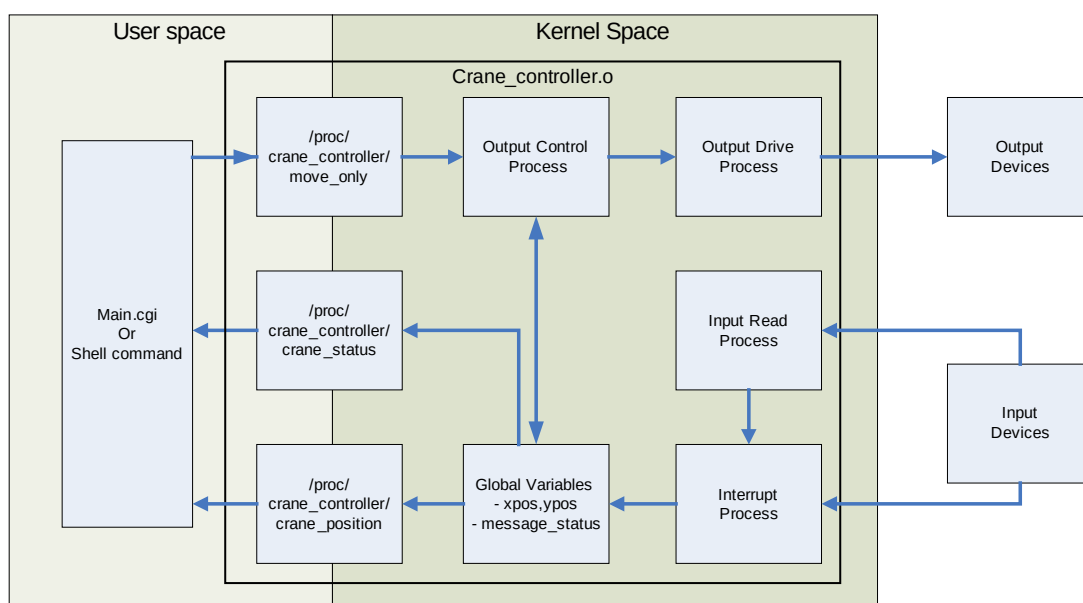
3.3.2 การออกแบบโปรแกรม

3.3.2.1 คุณสมบัติของโปรแกรม

- โปรแกรมจะถูกเขียนในลักษณะโมดูลเพื่อสามารถทำงานในระดับเคอร์เนลได้
- เมื่อเปิดเครื่องโปรแกรมจะต้องถูกติดตั้งโดยอัตโนมัติ
- โปรแกรมจะมีระบบตรวจสอบการทำงานด้วยตัวเองและสามารถแก้ไขปัญหาเบื้องต้นได้
- การติดต่อระหว่าง user space และ kernel space จะกระทำผ่าน Process file

3.3.2.2 โครงสร้างของโปรแกรม

เนื่องจากโปรแกรมมีลักษณะเป็นโมดูลจึงมีโครงสร้างที่คาบเกี่ยวระหว่าง user space และ kernel space ของระบบปฏิบัติการ โดยส่วนที่อยู่ใน user space นั้นผู้ใช้จะสามารถมองเห็นได้โดยตรงและถูกใช้เป็นเส้นทางการส่งและรับข้อมูลระหว่างผู้ใช้กับโมดูลที่กำลังทำงานอยู่ โครงสร้างหลักๆ ของโปรแกรมสามารถแสดงได้ดังรูป



รูปที่ 3-3 แสดงลักษณะโครงสร้างของ Crane controller device driver

- **User interface (Main.cgi or shell command)**

ทำหน้าที่ติดต่อกับผู้ใช้งาน เป็นส่วนรับคำสั่งจากยูสเซอร์แล้วส่งข้อมูลไปยัง Process file ในทางกลับกันยังทำหน้าที่รับข้อมูลจาก Process file มาแสดงยังยูสเซอร์ อีกทางหนึ่งด้วย โดยช่องทางการใช้งานระบบสามารถทำได้ 2 ทางคือ Linux shell command และหน้าเว็บเพจ (Web page) main.cgi

- **/proc/crane_controller/move_only**

เป็น Process file ที่ถูกสร้างขึ้นขณะที่โมดูลถูกโหลด (Load) เข้าสู่เคอร์เนลทำหน้าที่เป็นส่วนรับค่าตำแหน่งของเครนจากผู้ใช้และส่งค่าตำแหน่งต่อไปยังส่วนควบคุมการหมุนของ สเต็ปป์มอเตอร์ภายในของโปรเซสต่อไป

- **/proc/crane_controller/crane_position**

เป็น Process file ที่ถูกสร้างขึ้นขณะที่โมดูลถูกโหลดเข้าสู่เคอร์เนล ทำหน้าที่นำค่าตัวแปรโกลบอล (Global) ตำแหน่งของเครนไปแสดงยัง user space

- **/proc/crane_controller/crane_status**

เป็น Process file ที่ถูกสร้างขึ้นขณะที่โปรแกรมทำงาน ทำหน้าที่นำค่าตัวแปรโกลบอลของสถานะเครนไปแสดงยัง user space

- **output control process**

เป็นโปรเซสที่อยู่ภายในโมดูลทำหน้าที่รับคำสั่งให้สเต็ปป์มอเตอร์หมุนนำเครนไปยังตำแหน่งที่ผู้ใช้ต้องการ ซึ่งโปรเซสนี้จะเป็นตัวควบคุมทิศทางการหมุนตลอดจนความเร็วของสเต็ปป์มอเตอร์ทั้งหมด โดยจะทำการตรวจสอบค่าตำแหน่งของเครนจากตัวแปรโกลบอลตลอดเวลาเพื่อที่จะทราบว่าเครนไปถึงตำแหน่งที่ต้องการแล้วหรือยัง

- **output drive process**

เป็นส่วนที่ทำหน้าที่จัดตำแหน่งข้อมูลที่จะส่งให้กับสเต็ปป์มอเตอร์ โดยจะใช้หลักการของการมาสกิ้งบิต (Masking bit) ข้อมูลเพื่อกำหนดให้ค่าส่งไปยังสเต็ปป์มอเตอร์ได้ถูกตัว และหมุนได้อย่างถูกต้อง โดยจะมีการอ่านค่าการหมุนจากสเต็ปเทเบิล (Step table) ในส่วนของตัวแปรโกลบอล

- **input read process**

เป็นส่วนที่ทำหน้าที่รับสัญญาณดิจิทัลจากอุปกรณ์ภายนอกอย่างต่อเนื่องโปรเซสนี้จะต้องทำงานสอดคล้องกับอินเทอร์รัพโปรเซส (Interrupt process) เพราะเมื่อมีการเปลี่ยนแปลงของสัญญาณอินพุต (Input) สัญญาณอินเทอร์รัพ (Interrupt) จะต้องเปลี่ยนแปลงด้วย

- **interrupt process**

เป็นส่วนที่ทำหน้าที่ทำงานตามสัญญาณอินเทอร์รัพที่เกิดขึ้นจากอุปกรณ์ภายนอก โดยสัญญาณอินเทอร์รัพจะเกิดขึ้นพร้อมๆกับการเปลี่ยนแปลงของข้อมูลอินพุต ดังนั้นเมื่อเกิดอินเทอร์รัพขึ้นจะต้องมีการอ่านค่าจาก input read process เพื่อที่จะทราบว่าป็นสัญญาณอินเทอร์รัพของข้อมูลใด

3.3.2.3 จำนวนและขนาดของข้อมูลที่ใช้งาน

รายละเอียดของสัญญาณอินพุต

● TOP limit switch	1	Bit
● BOTTOM limit switch	1	Bit
● LEFT limit switch	1	Bit
● RIGHT limit switch	1	Bit
● CRANE Column sensor	1	Bit
● CRANE ROW sensor	1	Bit

รายละเอียดของสัญญาณเอาต์พุต (Output)

● X STEPPER MOTOR Drive	4	Bit
● Y STEPPER MOTOR Drive	4	Bit

รายละเอียดของพอร์ตขนานที่ใช้

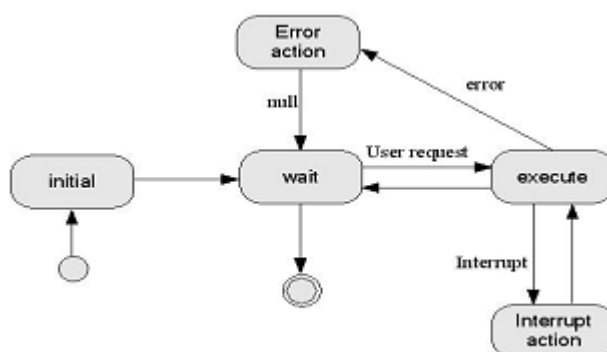
ขาที่ใช้รับข้อมูลอินพุต

● Bit 0	crane coulumn sensor
● Bit 1	crane row sensor
● Bit 2	not use
● Bit 3	not use
● Bit 4	right limit switch
● Bit 5	top limit switch
● Bit 6	left limit switch
● Bit 7	bottom limit switch

ขาที่ใช้ส่งข้อมูลเอาต์พุต

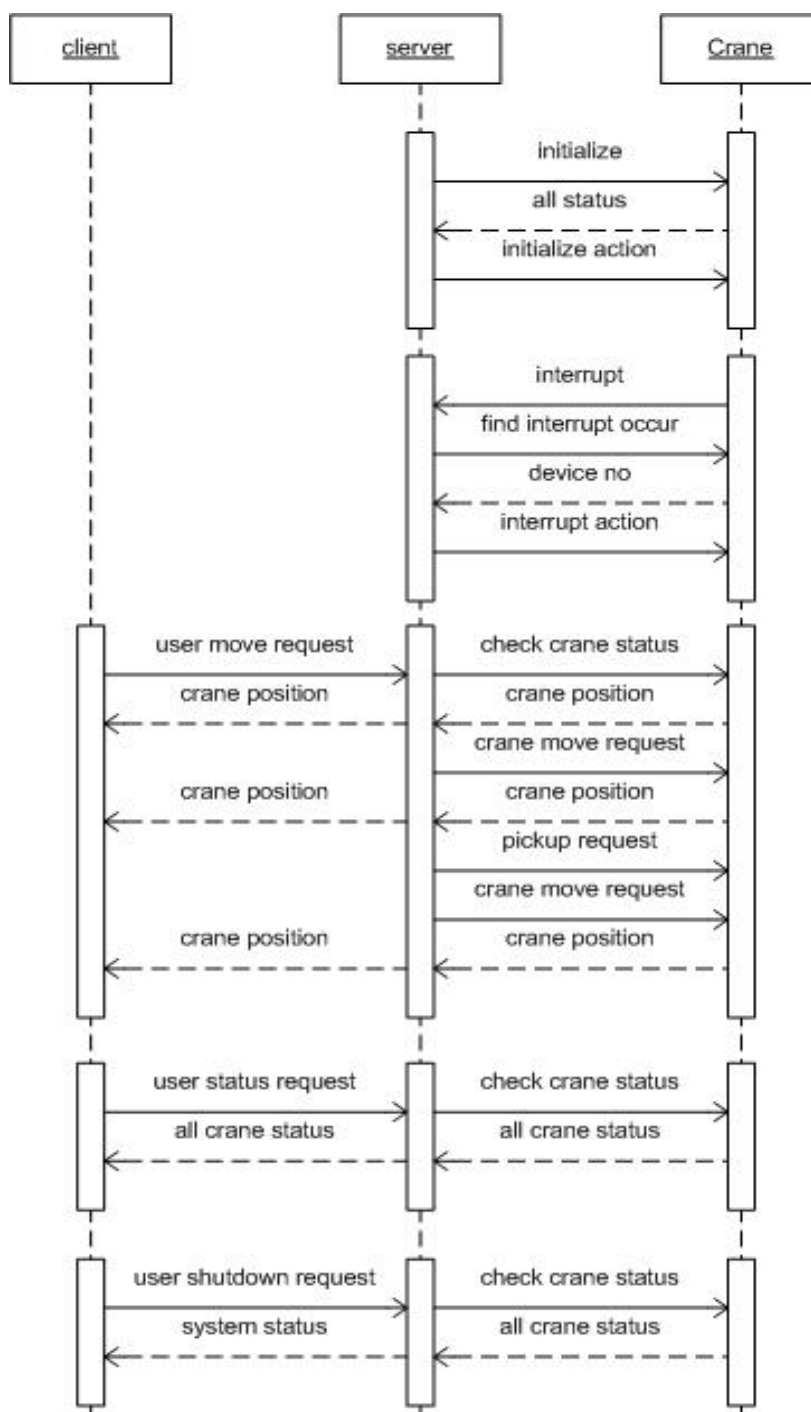
● Bit 0-3	x motor
● Bit 4-7	y motor

3.3.2.4 state diagram



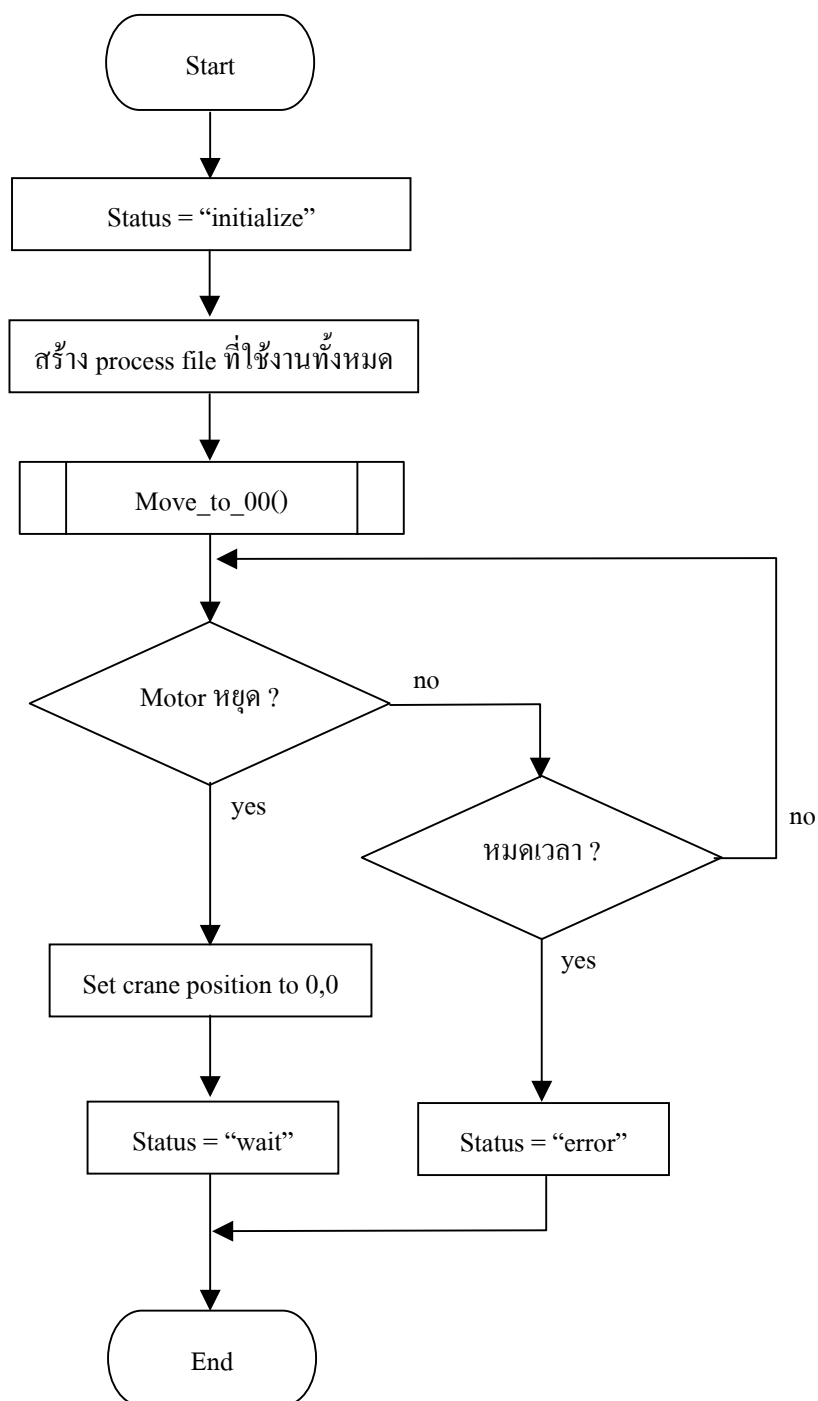
รูปที่ 3-4 แสดง state diagram ของระบบ

3.3.2.5 sequence diagram

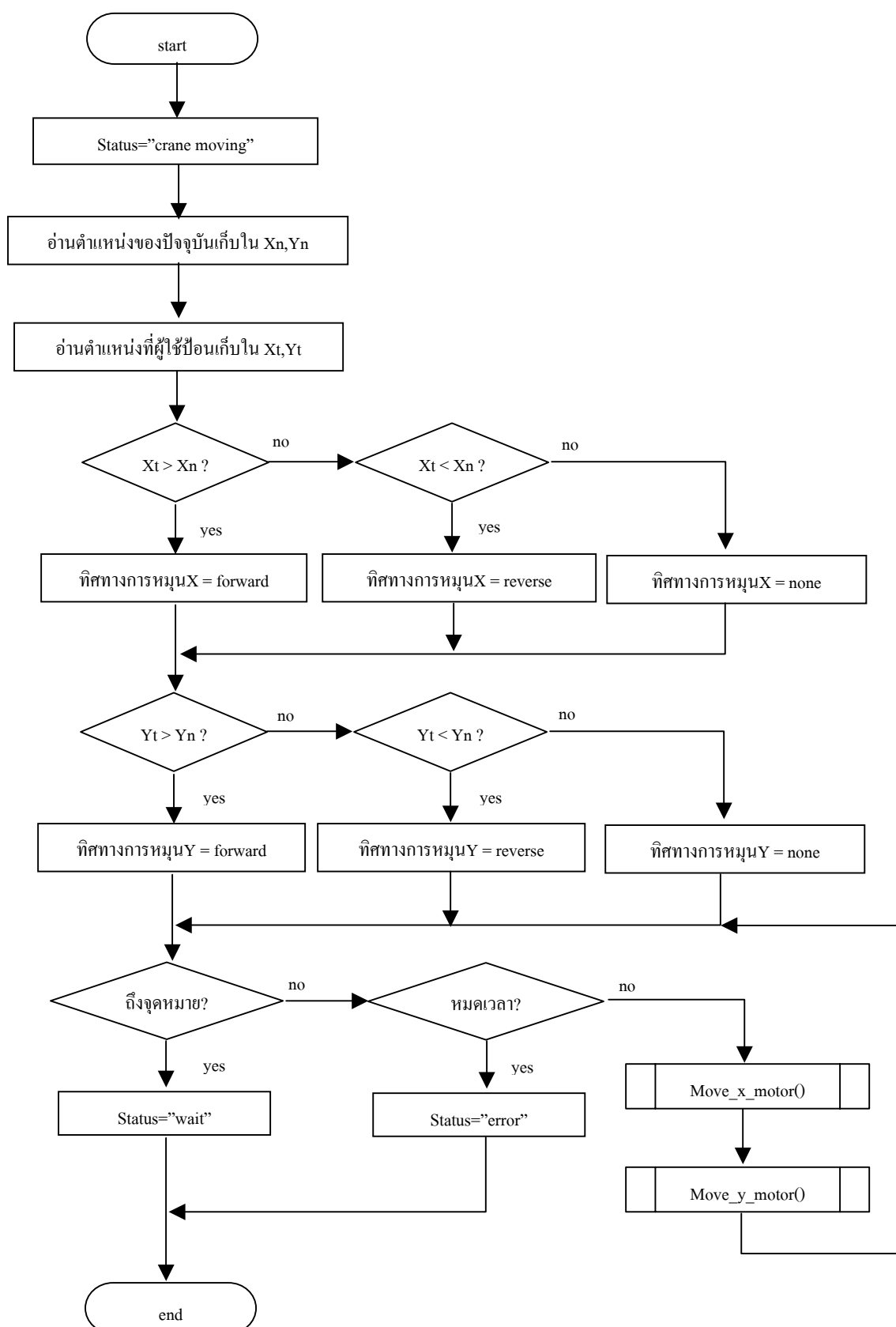


รูปที่ 3-5 แสดง sequence diagram ของระบบ

3.3.2.6 flowchart

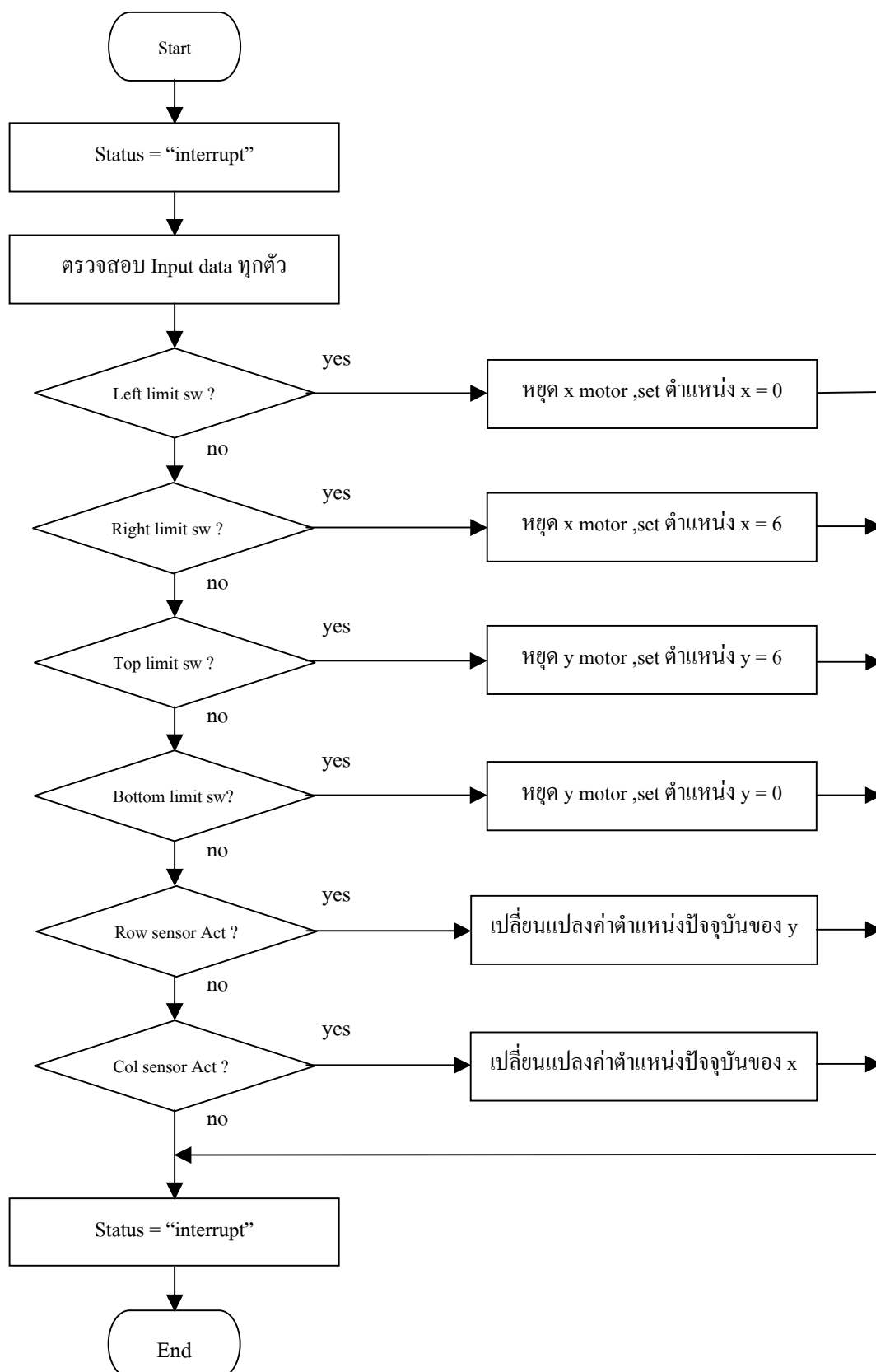
รูปที่ 3-6 แสดงขั้นตอนการทำงานของฟังก์ชัน `init_module()`

3.3.2.7 Crane_move procedure



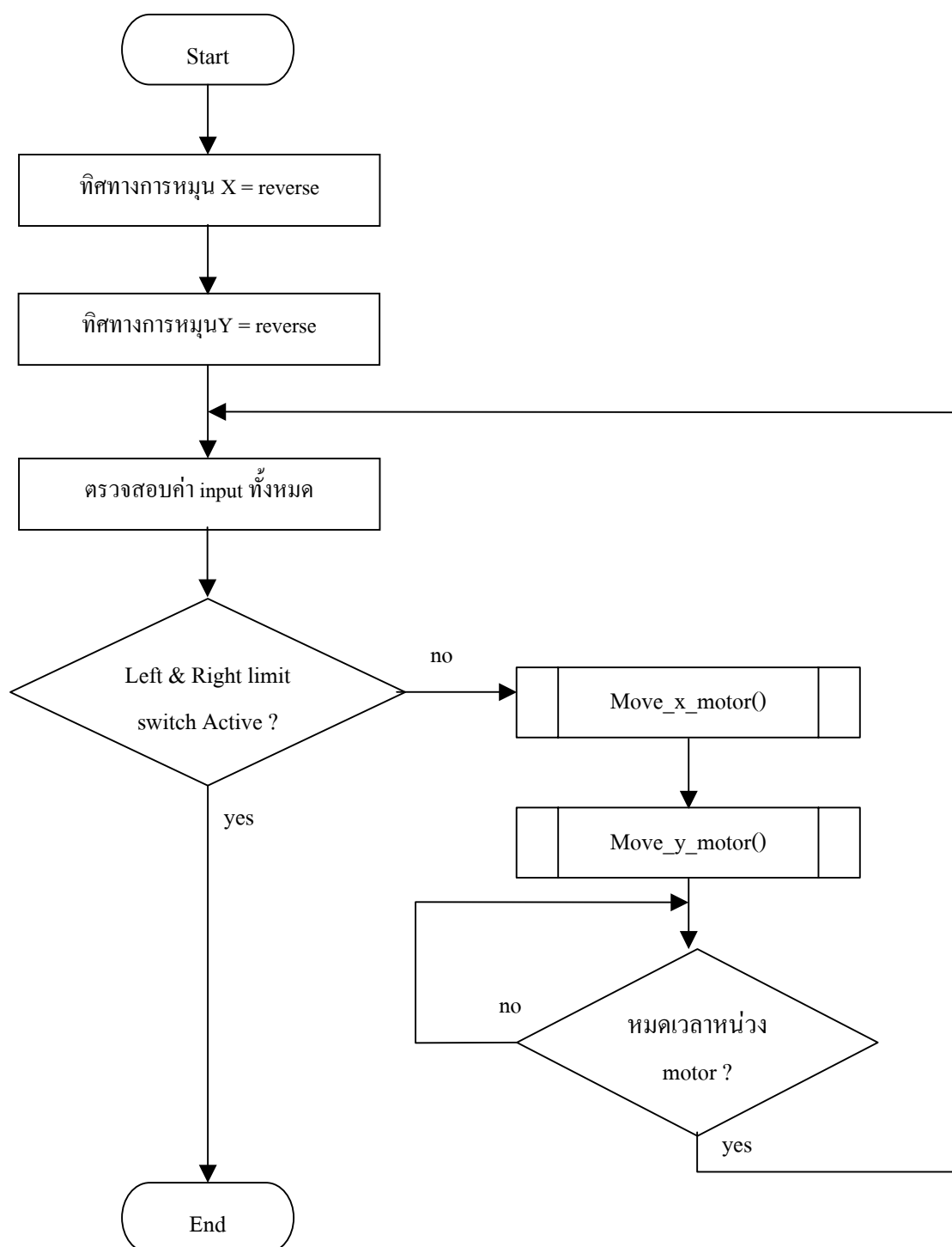
รูปที่ 3-7 แสดงขั้นตอนการทำงานเมื่อมีการสั่งให้เครนเคลื่อนที่

3.3.2.8 Interrupt_action procedure



รูปที่ 3-8 แสดงขั้นตอนการทำงานเมื่อมีอินเตอร์รัพ

3.3.2.9 Move to 0,0 position



รูปที่ 3-9 แสดงขั้นตอนการทำงานของ Move_to_00 ()

3.3.3 การเลือก Embedded Linux server

เนื่องจากการทำงานแบบ Embedded นั้น ไม่ได้กำหนดประเภทหรือขนาดของเมนบอร์ด (Mainboard) ที่ใช้ในการทำงาน ดังนั้นทางผู้จัดทำจึงได้เลือกเมนบอร์ดคอมพิวเตอร์ตระกูล x86 เพราะ

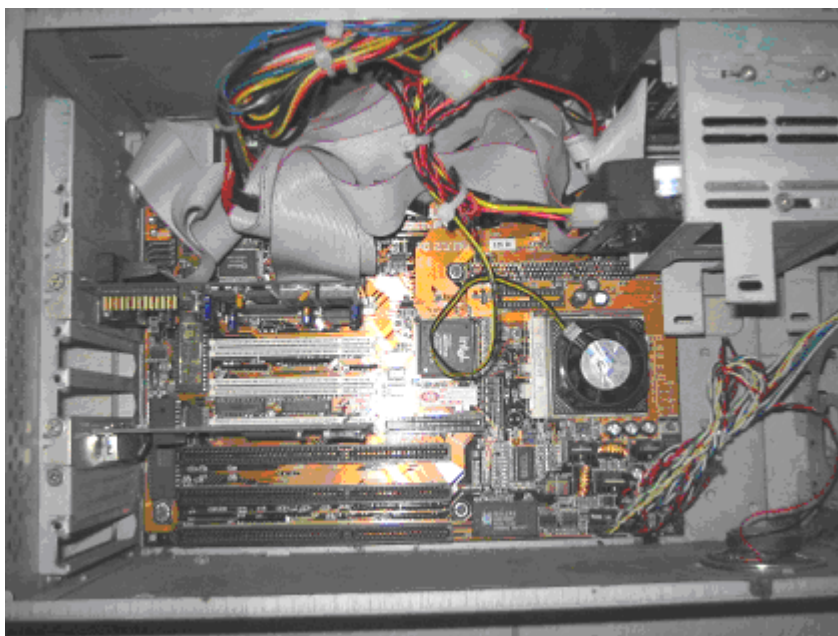
สามารถมีคุณสมบัติที่ครอบคลุมกับการทำงานของระบบและสามารถจัดหาได้ง่าย และเพื่อความสะดวกในการทำงาน โดยคุณสมบัติของเครื่องคอมพิวเตอร์ที่ผู้จัดทำเลือกใช้มีรายละเอียดดังนี้

3.3.3.1 Hardware Specifications

- CPU Pentium 200 MMX
- Memory 64 MB
- Hard Disk ขนาด 8 GB
- Card LAN 10/100
- VGA Card PCI

3.3.3.2 Software Specifications

- Debian Linux v. 3.0 r2



รูปที่ 3-10 แสดงเครื่องคอมพิวเตอร์ที่ทำหน้าที่ *Embedded Linux Server*

3.3.4 การออกแบบวงจรเรณอินเตอร์เฟส

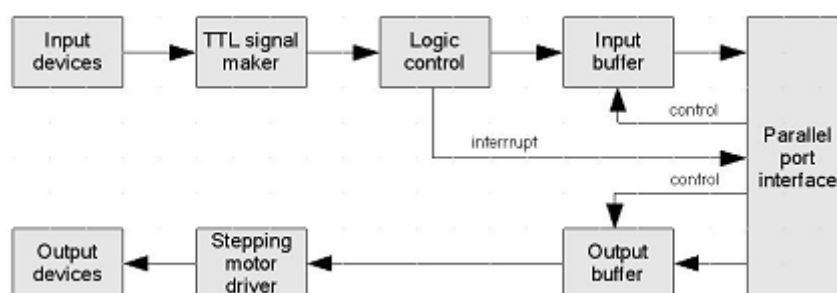
เรณอินเตอร์เฟสเป็นวงจรอิเล็กทรอนิกส์ (Electronic) ที่ทำหน้าที่ติดต่อระหว่างเซิร์ฟเวอร์กับอุปกรณ์ที่ติดตั้งบนเรณทั้งหมด เนื่องจากอุปกรณ์ภายนอกจะใช้ระดับสัญญาณแตกต่างจากสัญญาณดิจิทัลที่ใช้ในคอมพิวเตอร์ เช่น สเต็ปมิ่งมอเตอร์ต้องใช้แรงดันขนาด 12 โวลต์ (V) ในการขับเคลื่อนแต่ละขด หรืออุปกรณ์เซ็นเซอร์ (Sensor) ต่างๆ ที่เป็น dry contact switch จะต้องใช้แรงดัน 5 โวลต์ ในการ

สร้างสัญญาณที่ที่แอลเพื่อป้อนให้กับอินพุตของโปรแกรม ดังนั้นวงจรส่วนนี้จึงทำหน้าที่เหมือนเป็นตัวปรับแรงดันของสัญญาณอินพุตนั่นเอง

3.3.4.1 คุณสมบัติของเกรนอินเตอร์เฟส

- รับส่งข้อมูลระหว่างเครื่องคอมพิวเตอร์ผ่านทางพอร์ตขนาน
- ใช้พอร์ตข้อมูล (Data port) 8 บิตของพอร์ตขนานในการรับและส่งข้อมูล (data)
- การรับอินพุตและเอาต์พุตสามารถทำได้โดยส่งคำสั่งควบคุมมายังพอร์ตควบคุม (Control port) ของพอร์ตขนาน
- สามารถสร้างสัญญาณอินเตอร์รัพ (Interrupt signal) เป็นเวลาสั้นๆ (ประมาณ 100-200 ns) เพื่อกระตุ้นให้โปรแกรมทำงานช่วงสั้นๆ
- เมื่อเกิดอินเตอร์รัพข้อมูลอินพุตจะต้องปรากฏบนพอร์ตข้อมูลอย่างต่อเนื่องหรือจนกระทั่งโปรแกรมอ่านค่าเรียบร้อยแล้ว
- สามารถส่งข้อมูลเอาต์พุตให้กับสเต็ปมิ่งมอเตอร์ ได้โดยตรงและต่อเนื่อง
- ต้องมีหัวเสียบสำหรับรับอินพุตจากชุดเกรน

3.3.4.2 crane interface block diagram



รูปที่ 3-11 แสดง Block diagram ของวงจรเกรนอินเตอร์เฟส

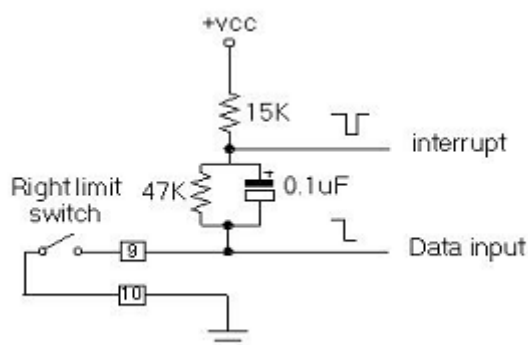
3.3.4.3 การออกแบบวงจร

จากบล็อกไดอะแกรมสามารถนำมาเขียนวงจรตามคุณสมบัติที่ต้องการได้ดังนี้

- วงจรรับสัญญาณ dry contact

สิ่งที่ทราบว่าสวิตช์ (switch) หรืออุปกรณ์ที่ไม่มีไฟเลี้ยงอื่นๆ ต้องการแรงดันในการทำงาน ถ้าเป็นการใช้งาน input data ทั่วๆไปอาจใช้ตัวต้านทานเพียงตัวเดียวในการทำงาน แต่เนื่องจากผู้จัดทำต้องการสร้างพัลส์ (pulse) เพื่อใช้ในการกระตุ้นสัญญาณอินเตอร์รัพจึงนำคาปาซิเตอร์ (Capacitor) มาต่อเพิ่มเติมเพื่ออาศัยช่วงเวลาเวลาที่คาปาซิเตอร์กำลังชาร์จประจุสร้างพัลส์ขึ้นและเมื่อคาปาซิเตอร์ชาร์จประจุจนเต็มสัญญาณอินเตอร์รัพจะหยุดโดยอัตโนมัติ

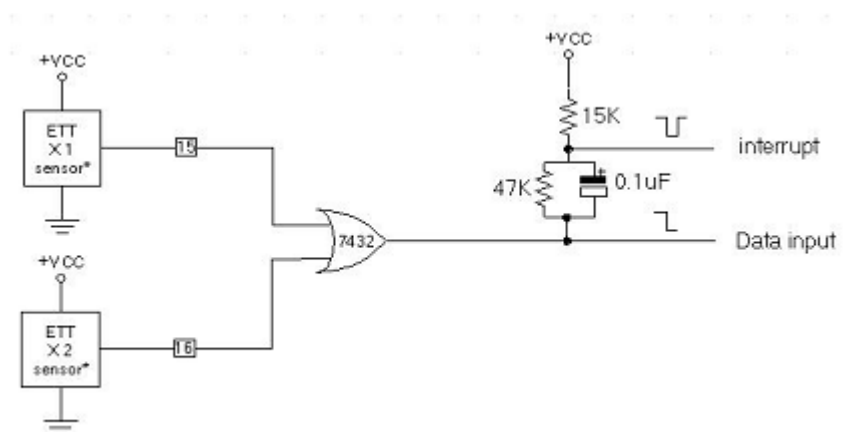
ส่วนหน้าที่ของรีซิสเตอร์ 47K คือใช้ในการคายประจุ (discharge) ประจุออกจากคาปาซิเตอร์กรณีที่สวิตช์ถูกอยู่ในสภาวะออฟ (OFF) แล้ว



รูปที่ 3-12 แสดงวงจรรับสัญญาณ dry contact จากอุปกรณ์ภายนอก

- วงจรรับสัญญาณจากอุปกรณ์เซ็นเซอร์

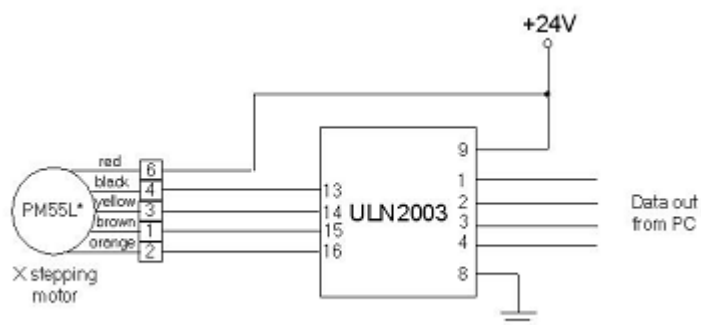
เนื่องจากอุปกรณ์เซ็นเซอร์ต่างๆ ส่วนใหญ่ใช้วงจรออปแอมป์ (OP AMP) เป็นตัวตรวจวัดความแตกต่าง จึงสามารถที่จะจ่ายแรงดันให้กับคอมพิวเตอร์ได้โดยตรง แต่จะใช้หลักการเดียวกับวงจรรับสัญญาณ dry contact เพื่อสร้างสัญญาณ interrupt ดังรูป



รูปที่ 3-13 แสดงวงจรรับสัญญาณจากอุปกรณ์เซ็นเซอร์ภายนอก

- วงจรขับสเต็ปมอเตอร์

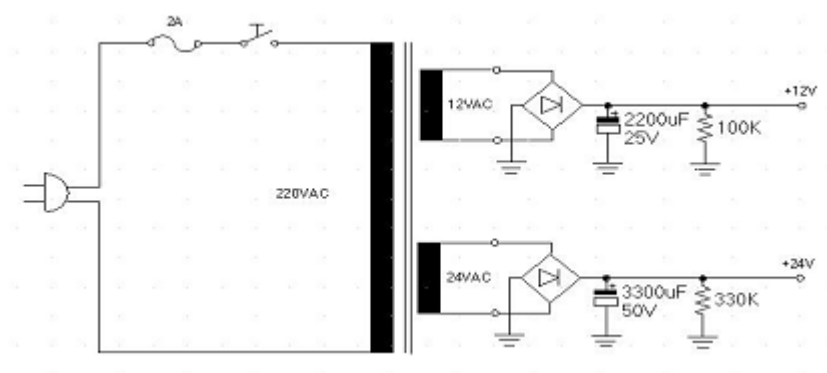
เนื่องจากแรงดันที่จ่ายออกมาจากเครื่องคอมพิวเตอร์เป็นแรงดันระดับต่ำประมาณ 5 โวลต์ การที่จะนำไปใช้ขับสเต็ปมอเตอร์ได้จึงต้องมีการปรับแรงดันให้เหมาะสมกับความต้องการของสเต็ปมอเตอร์ โดยใช้ไอซี (IC) ULN2003 เป็นตัวจัดการเรื่องนี้โดยสามารถคุณสมบัตินี้ได้จากภาคผนวก ง



รูปที่ 3-14 แสดงวงจรการปรับระดับสัญญาณที่ส่งไปยังสเต็ปปีงมอเตอร์

- วงจรจ่ายแรงดัน

เนื่องจากการทดสอบการทำงานของสเต็ปปีงมอเตอร์ ได้มีการทดสอบกับแรงดันทั้ง 24 โวลต์ และ 12 โวลต์ เพื่อทดสอบแรงบิดของมอเตอร์ (motor) และตรวจสอบปริมาณความร้อนที่สะสมบนตัวมอเตอร์ ดังนั้นจึงต้องออกแบบแรงดันขับเคลื่อนมอเตอร์ไว้ 2 ระดับดังรูป



รูปที่ 3-15 แสดงวงจรเพาเวอร์ซัพพลายเพื่อจ่ายไฟให้กับสเต็ปปีงมอเตอร์

หมายเหตุ วงจรที่สมบูรณ์สามารถดูได้จากภาคผนวก จ

3.3.5 การออกแบบเคาน์เตอร์และโครง

เคาน์เตอร์และโครงเป็นส่วนโครงสร้างการทำงานทางกลของเคาน์เตอร์ทั้งหมด โครงสร้างทั้งหมดถูกสร้างขึ้นจากอะลูมิเนียม โดยเคาน์เตอร์และโครงที่ต้องการมีคุณสมบัติดังต่อไปนี้

3.3.5.1 คุณสมบัติของเคาน์เตอร์และโครง

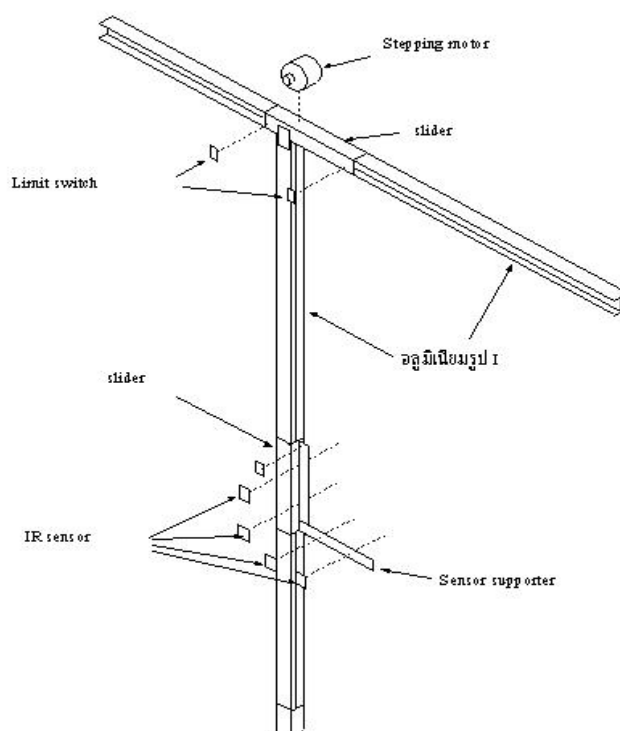
- ตัวเคาน์เตอร์สามารถเคลื่อนไหวได้ทั้งแนวตั้งและแนวนอนด้วยสเต็ปปีงมอเตอร์จำนวน 2 ตัว
- โครงมีขนาดประมาณ 80 cm. x 80 cm. มีจำนวนช่องเท่ากับ 5x5 ช่องแต่ละช่องมีขนาด 16 cm. x 18 cm. ขอบของช่องแต่ละช่องมีความกว้างประมาณ 1 cm. เพื่อใช้ในการตรวจจับของ

เซ็นเซอร์และโครงสร้างของโครงสามารถวางได้อย่างแข็งแรงและมั่นคง

- มีลิมิตสวิตช์ (limit switch) สำหรับตรวจจับการเคลื่อนที่ของเครนทั้ง 4 ด้านคือ ซ้าย, ขวา, บน และล่าง
- มีตัวตรวจจับตำแหน่งเครน 4 ตัว ติดตั้งที่เครนทั้ง 2 แนวโดยแต่ละแนวจะใช้ตัวเซ็นเซอร์แนวละ 2 ตัว เพื่อตรวจสอบความถูกต้องของสัญญาณที่ส่งให้กับ interface card
- ต้องมีการสร้างคอนเน็คเตอร์ (connector) สำหรับเชื่อมต่อขาสัญญาณอินพุตและเอาต์พุตทั้งหมดไปยังส่วน crane interface card

3.3.5.2 การออกแบบโครงสร้างของเครน

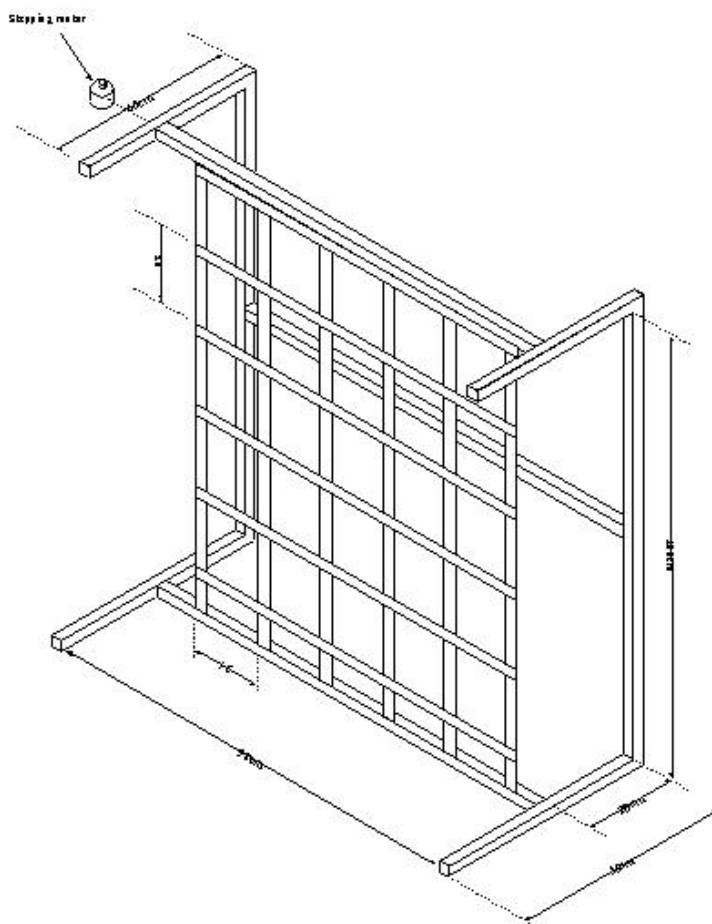
ในการออกแบบโครงสร้างของเครน ไม่มีรูปแบบในการสร้างที่แน่นอนขึ้นอยู่กับความสะดวกในการจัดหาวัสดุ และการเลือกโครงสร้างที่ไม่ซับซ้อนที่สามารถเคลื่อนไหวได้ง่ายและคงที่ จากการศึกษาการทำงานของระบบเครนประเภทต่างๆ ได้ข้อสรุปว่าเครนควรมีลักษณะคล้ายแขนของเครื่องพล็อตเตอร์ (plotter) โดยส่วนที่เคลื่อนที่จะอยู่ในแนวคังเพื่อลดปัญหาเนื่องจากแรงโน้มถ่วง



รูปที่ 3-16 แสดงลักษณะโครงสร้างของเครน

3.3.5.3 การออกแบบโครงสร้างของโครง

ปัญหาของการออกแบบโครงสร้างของโครงอยู่ที่ทำอย่างไรให้โครงแข็งแรงและมั่นคง โดยใช้โครงสร้างที่ซับซ้อนน้อยที่สุด จากการทดลองและทดสอบโครงสร้างหลายรูปแบบ ในที่สุดผู้จัดทำได้ข้อสรุปเกี่ยวกับรูปแบบของโครงสร้างที่มีความแข็งแรงและไม่ซับซ้อน ดังนี้



รูปที่ 3-17 แสดงลักษณะโครงสร้างของโครง

3.4 การติดตั้งและปรับแต่งเซิร์ฟเวอร์

3.4.1 การติดตั้ง Debian Linux

Debian linux v.3.0 r2 เป็นแผ่นติดตั้งที่สามารถเลือกได้ว่าจะติดตั้งด้วย kernel 2.2 หรือ kernel 2.4 โดยในการทดลองนี้เลือกที่จะพัฒนาด้วย kernel 2.4 ด้วยเหตุผลที่ว่ามีความทันสมัยและใช้งานง่ายกว่า kernel 2.2 ถึงแม้ว่าในขณะที่ทำการทดลองนี้จะมี kernel 2.6 ออกสู่ตลาดแล้วก็ตาม แต่ด้วยเหตุที่มีเอกสารสนับสนุนน้อยจึงเลือกที่จะทำงานกับ kernel 2.4 ต่อไป

การติดตั้ง Debian Linux ด้วย kernel 2.4 สามารถทำได้โดยพิมพ์คำสั่ง bf24 หลังจากบูตเครื่องคอมพิวเตอร์ด้วยแผ่นซีดีแล้วและในส่วนรายละเอียดของขั้นตอนการติดตั้งสามารถดาวน์โหลดได้ที่ <http://www.debian.org/releases/stable/i386/install.en.pdf>

3.4.2 การปรับแต่งเซิร์ฟเวอร์

การปรับแต่งเซิร์ฟเวอร์หลังการติดตั้งเป็นสิ่งจำเป็นเช่นเดียวกับการติดตั้งระบบ เพราะการเขียนโปรแกรมหรือการทดสอบการทำงานผ่านเว็บเพจอาจจะทำไม่ได้ถ้าไม่ตรวจสอบการทำงานของเซิร์ฟเวอร์ให้สมบูรณ์ โดยจุดที่ต้องตรวจสอบมีดังนี้

3.4.2.1 การตรวจสอบเวอร์ชันของ linux kernel และ linux include file

เนื่องจากการพัฒนาโปรแกรมบนลินุกซ์ในระหว่างรันโปรแกรม ระบบปฏิบัติการจะตรวจสอบเวอร์ชันของโปรแกรมที่กำลังทำงานด้วยว่าเป็นเวอร์ชันเดียวกับเคอร์เนลหรือไม่ ดังนั้นการตรวจสอบเวอร์ชันของเคอร์เนลและ include file จึงเป็นสิ่งจำเป็นสามารถตรวจสอบได้โดย

- ตรวจสอบเวอร์ชันของ linux kernel โดยพิมพ์คำสั่ง

```
uname -a
```

```
Linux project 2.4.18 #2 SMP Thu Sep 30 07:15:47 ICT 2004 i586 unknown
```

- ตรวจสอบเวอร์ชันของ include file โดยพิมพ์คำสั่ง

```
cat /usr/include/linux/version.h
```

```
#define UTS_RELEASE "2.4.18"
```

```
#define LINUX_VERSION_CODE 132114
```

```
#define KERNEL_VERSION(a,b,c) (((a) << 16) + ((b) << 8) + (c))
```

หมายเหตุ ในส่วนที่ขีดเส้นใต้จะต้องตรงกันทั้ง 2 จุดถ้าไม่ตรงกันให้ทำการแก้ไขค่าในไฟล์ /usr/include/linux/version.h ให้ตรงกับเวอร์ชันของเคอร์เนลที่ใช้อยู่

3.4.2.2 การตรวจสอบการทำงานของเน็ตเวิร์กการ์ด

ในเบื้องต้นจะต้องตรวจสอบว่าเน็ตเวิร์กการ์ดทำงานอยู่และ กำหนดหมายเลขไอพีอย่างถูกต้องหรือไม่ ซึ่งหมายเลขไอพีนั้นผู้ใช้งานสามารถเปลี่ยนแปลงและกำหนดใหม่ได้ แต่ควรสอดคล้องกับ Client ที่ใช้ในการพัฒนาโปรแกรม ตามตัวอย่างได้มีการกำหนดหมายเลขไอพีไว้ที่ 192.168.0.50 คำสั่งที่ใช้ตรวจสอบหมายเลขไอพีคือ ifconfig หน้าจอจะแสดงรายชื่อของเน็ตเวิร์กการ์ดที่กำลังทำงานอยู่ทั้งหมด

```
eth0  Link encap:Ethernet HWaddr 00:50:04:BA:99:34
      inet addr:192.168.0.50 Bcast:192.168.0.255 Mask:255.255.255.0
      UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
      RX packets:961 errors:0 dropped:0 overruns:0 frame:0
      TX packets:619 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:100
      RX bytes:59476 (58.0 KiB) TX bytes:58146 (56.7 KiB)
      Interrupt:11 Base address:0xe000

lo    Link encap:Local Loopback
      inet addr:127.0.0.1 Mask:255.0.0.0
      UP LOOPBACK RUNNING MTU:16436 Metric:1
      RX packets:83 errors:0 dropped:0 overruns:0 frame:0
```

```
TX packets:83 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:0
RX bytes:5815 (5.6 KiB) TX bytes:5815 (5.6 KiB)
```

ผู้ใช้ควรให้ความสนใจเฉพาะเน็ตเวิร์กการ์ด eth0 เพราะ lo เป็นโลคอลอินเทอร์เฟซ (local interface) ของระบบ ถ้าปรากฏข้อความดังตัวอย่างด้านบนแสดงว่าเน็ตเวิร์กการ์ดถูกติดตั้งและทำงานอย่างสมบูรณ์แล้ว แต่ถ้าไม่เป็นไปตามที่กล่าวมาให้ทำการตรวจสอบค่าภายในไฟล์ `/etc/network/interfaces` ภายในไฟล์ควรประกอบด้วย

```
# /etc/network/interfaces -- configuration file for ifup(8), ifdown(8)

# The loopback interface
auto lo eth0

iface lo inet loopback

iface eth0 inet static
address 192.168.0.50
netmask 255.255.255.0
```

หมายเหตุ ส่วนที่ขีดเส้นใต้เป็นหมายเลขไอพีของเซิร์ฟเวอร์เมื่อแก้ไขเรียบร้อยแล้วให้ทำการจัดเก็บ (save) และพิมพ์คำสั่ง `/etc/init.d/network restart` เพื่อให้ระบบเน็ตเวิร์กทำงานใหม่อีกครั้ง

3.4.2.3 การตรวจสอบการทำงานของเทลเน็ตและเอฟทีพี

Connected to project.

220 project FTP server (Version 6.4/OpenBSD/Linux-ftpd-0.17) ready.

Name (localhost:root):

จะต้องปรากฏสื่ออินพรีม (login prompt) ดังตัวอย่างด้านบนและเช่นเดียวกับเอฟทีพีในการตรวจสอบ telnet service สามารถทำได้เช่นเดียวกัน โดยพิมพ์คำสั่ง `telnet localhost` จะต้องปรากฏข้อความดังนี้

Trying 127.0.0.1...

Connected to project.

Escape character is '^['.

Debian GNU/Linux 3.0 project

project login:

ข้อความด้านบนจะถูกแสดงในกรณีที่เทลเน็ตและเอฟทีพีกำลังทำงานอยู่ถ้าไม่เป็นไปตามตัวอย่าง ให้ทำการตรวจสอบไฟล์ `/etc/inetd.conf` ว่าทำการเปิดให้บริการเทลเน็ตและเอฟทีพีหรือยัง โดยภายใน `inetd.conf` ควรจะเป็นดังนี้

```
# /etc/inetd.conf: see inetd(8) for further informations.
```

```
#:STANDARD: These are standard services.
```

```
ftp      stream  tcp      nowait  root            /usr/sbin/tcpd  /usr/sbin/in.ftpd
telnet   stream  tcp      nowait  telnetd.telnetd /usr/sbin/tcpd  /usr/sbin/in.telnetd
```

สังเกตว่าหน้าบรรทัดที่มีข้อความเทลเน็ตและเอฟทีพีจะต้องไม่มีเครื่องหมาย “#” อยู่ด้านหน้า ถ้ามีให้ทำการลบเครื่องหมาย “#” ออก จากนั้นทำการจัดเก็บและรีสตาร์ทระบบใหม่และทำการตรวจสอบรายละเอียดดังที่กล่าวมาแล้วอีกครั้ง ถ้ายังไม่สามารถใช้งานได้ให้ตรวจสอบว่าเทลเน็ตและเอฟทีพีได้ถูกติดตั้งหรือยังโดยการตรวจสอบแพ็คเกจที่ติดตั้งไว้ดังนี้

```
dpkg -l |grep ftp
```

```
ii      ftp      0.17-9      The FTP client.
ii      ftpd      0.17-13     FTP server
ii      lftp      2.4.9-1     Sophisticated command-line FTP/HTTP client p
```

```
dpkg -l |grep telnet
```

```
ii      telnet   0.17-18     The telnet client.
li      telnetd   0.17-18     The telnet server.
```

ถ้าไม่เป็นไปตามตัวอย่างด้านบนที่กล่าวมาให้ทำการติดตั้งแพ็คเกจหมดโดยถ้าเซิร์ฟเวอร์สามารถเชื่อมต่ออินเทอร์เน็ตได้ให้ทำการพิมพ์คำสั่ง `apt-get install ftp` เพื่อติดตั้งการให้บริการเอฟทีพีและ `apt-get install telnet` เพื่อติดตั้งการให้บริการเทลเน็ตระบบจะทำการดาวน์โหลดและติดตั้งแพ็คเกจโดยอัตโนมัติ

3.4.2.4 การตรวจสอบการทำงานของเว็บเซิร์ฟเวอร์

การทำงานของเว็บเซิร์ฟเวอร์ (Web server) หรือ Apache สามารถตรวจสอบการทำงานได้โดยใช้คำสั่ง `ps ax|grep apache` จะปรากฏข้อความดังนี้

```
198 ?    S    0:00 /usr/sbin/apache
312 ?    S    0:00 [apache]
313 ?    S    0:00 [apache]
314 ?    S    0:00 [apache]
```

```
315 ?    S    0:00 [apache]
316 ?    S    0:00 [apache]
927 pts/0  S    0:00 grep apache
```

ถ้าไม่มีข้อความที่คล้ายคลึงกับตัวอย่างด้านบนให้ทำการตรวจสอบว่า apache ถูกติดตั้งแล้วหรือยัง โดยใช้คำสั่ง `dpkg -l |grep apache` จะปรากฏข้อความดังนี้

```
ii apache                1.3.26-0woody3 Versatile, high-performance HTTP server
ii apache-common         1.3.26-0woody3 Support files for all Apache web servers
```

ถ้าไม่ปรากฏข้อความดังตัวอย่าง ให้ทำการติดตั้ง apache โดยใช้คำสั่ง `apt-get install apache` หรือติดตั้งจากแผ่นซีดี Debian linux แผ่นที่ 1 ด้วยคำสั่ง `dpkg -i` ตามด้วยชื่อไฟล์ apache

การคอมไพล์เคอร์เนลเป็นส่วนการปรับแต่งเพิ่มเติมอาจต้องทำให้รองรับการทำงานเมนบอร์ดรุ่นอื่นๆ แต่จากการทดสอบพบว่าการนำ Debian Linux kernel 2.4 ไปใช้งานกับเมนบอร์ดตระกูล x86 ไม่จำเป็นต้องคอมไพล์เคอร์เนลใหม่ โดยสามารถใช้งานได้เป็นปกติ หากผู้ใช้ต้องการคอมไพล์เคอร์เนลใหม่สามารถดูรายละเอียดได้จาก www.kernel.org

3.5 การเขียนโปรแกรม

3.5.1 โปรแกรมส่วน Crane controller

ฮาร์ดโค้ดที่สมบูรณ์สามารถดูได้จากภาคผนวก ก. ในส่วนที่จะกล่าวต่อไปเป็นเพียงฮาร์ดโค้ดบางส่วนที่นำมาประกอบคำอธิบายการทำงานและเปรียบเทียบกับโครงสร้างโมดูลที่ทำการออกแบบไว้สามารถแบ่งฮาร์ดโค้ดเป็นส่วนๆ ได้ดังนี้

3.5.1.1 โปรแกรมส่วน Global declaration

```
static struct proc_dir_entry *main_dir,      /* main process directory */
*move_only_file,                             /* move motor only process */
*get_position_file,                          /* read crane position process */
*get_status_file;                           /* read all crane status process */
```

เป็นส่วนที่แสดงโครงสร้างระบบไฟล์ของโปรเซสที่ถูกสร้างขึ้นภายใต้ /proc directory จากโค้ดด้านบนจะแสดงให้เห็นโครงสร้างไคลเร้กทอรีจำนวน 1 ไคลเร้กทอรีประกอบด้วยไฟล์จำนวน 4 ไฟล์

```

/*****

all status messages

*****/

/* crane status */

char *init_string = "Initializing crane please wait\n";
char *standby_string = "Crane standby for your request\n";
char *moving_string = "Crane moving to specified position\n";
char *demo_string = "Program running in demo mode\n";
char *timeout_error_string = "Crane not reach to target position in time\n";
char *left_limit_string = "Left limit action\n";
char *right_limit_string = "Right limit action\n";
char *top_limit_string = "Top limit action\n";
char *bottom_limit_string = "Bottom limit action\n";

/* user warning message */

char *user_position_error = "Please enter X,Y position between 0,0 and 5,5 coordinate\n";
char *user_keyin_error = "Please enter number in format X,Y between 0,0 and 5,5 coordinate\n";

```

บอร์ดโค้ดด้านบนแสดงให้เห็นการกำหนดแมสเสดที่แจ้งให้กับผู้ใช้งานเมื่อระบบอยู่ในสถานะต่างๆ

```

/*****

/* define global variables here */

*****/

int xyposchange=1; /*used for protect x,y position if motor not moving */
int timeout = 0;          /*used for limit motor action */

long xmotorstarttime = 0; /* used for check x sensor interval time */
long ymotorstarttime = 0;

int xflag = 0;          /* start x sensor flag */
int yflag = 0;

int xflag2 = 0;

```

```

int yflag2 = 0;

int xpos=0;           /*crane x position counter */
int ypos=0;           /*crane y position counter */

char xdirect,ydirect;  /* global variable for motor direction */

char progmode = 'a';   /* a = actual , d = demo */

unsigned short motorout ;      /*variable for keep stepping motor data out */

unsigned char ystepable[MAXSTEP+1] = {0x03,0x06,0x0c,0x09};
unsigned char xstepable[MAXSTEP+1] = {0x90,0xc0,0x60,0x30};

int xindex=0;         /*stepper motor move index */
int yindex=0;

```

ส่วนของโปรแกรมด้านบนแสดงให้เห็นตัวแปรโกลบอลทั้งหมดที่ใช้ในการทำงาน

3.5.1.2 โปรแกรมส่วน Module initialize

ซอร์สโค้ดส่วนนี้จะถูกเรียกใช้งานเมื่อโมดูลถูกโหลดเข้าสู่เคอร์เนลการทำงานส่วนใหญ่จะเป็นการเตรียมพื้นที่สำหรับทำงานให้แก่โมดูล เช่นการตรวจสอบค่าหมายเลขอินเตอร์รัพว่าสามารถใช้งานได้หรือไม่ หรือสร้าง Directory Tree ให้กับ Process file ทั้งหมดเป็นต้น

```

static int __init init_crane_controller(void)
{
    int rv = 0;
    int i;

    /* Create the crane_controller /proc entry */
    main_dir = proc_mkdir("crane_controller", NULL);
    if(main_dir == NULL) {
        rv = -ENOMEM;
        goto out;
    }
}

```


ส่วนของโปรแกรมด้านบนเป็นการสร้างไดเรกทอรีหลักชื่อ crane_controller ภายใต้ /proc ไดเรกทอรี

```
main_dir->owner = THIS_MODULE;

/* Create liftacmains and make it readable by all - 0444 */
move_only_file = create_proc_entry("move_only", 0666, main_dir);
if(move_only_file == NULL) {
    rv = -ENOMEM;
    goto no_move_only;
}

move_only_file->read_proc = NULL;
move_only_file->write_proc = &proc_move_only;
move_only_file->owner = THIS_MODULE;
```

ส่วนของโปรแกรมด้านบนเป็นการสร้าง Process file ที่ทำหน้าที่รับข้อมูลจากผู้ใช้โดยไฟล์ที่สร้างขึ้นนี้จะรับค่าอินพุตจากผู้ใช้แล้วจะส่งค่าให้กับโปรแกรมย่อย proc_move_only

```
/*status file */
get_status_file = create_proc_entry("crane_status", 0444, main_dir);
if(get_status_file == NULL) {
    rv = -ENOMEM;
    goto no_get_status;
}

get_status_file->data = init_string;
get_status_file->read_proc = &proc_read_status;
get_status_file->write_proc = NULL;
get_status_file->owner = THIS_MODULE;
```

ส่วนของโปรแกรมด้านบนเป็นการสร้าง Process file ที่ทำหน้าที่ส่งข้อมูลไปยังผู้ใช้โดยไฟล์ที่สร้างขึ้นนี้จะส่งค่า status message ให้กับผู้ใช้โดยจะอ่านค่าจาก proc_read_status

```
crane_interrupt_file = create_proc_entry("crane_interrupt", 0444, main_dir);
if(crane_interrupt_file == NULL)
{
    return -ENOMEM;
}
```

```

crane_interrupt_file->read_proc = NULL;
crane_interrupt_file->write_proc = NULL;
crane_interrupt_file->owner = THIS_MODULE;

/* request interrupt from linux */
rv = request_irq(INTERRUPT, crane_interrupt_interrupt, 0,"crane_interrupt",NULL);
if ( rv ) {
    printk("Can't get interrupt %d\n", INTERRUPT);
    goto no_interrupt;
}

```

ส่วนของโปรแกรมด้านบนเป็นการสร้าง Process file ที่จะทำงานทุกครั้งที่เกิดอินเทอร์รัพจากอุปกรณ์ภายนอกโดยเมื่อเกิดอินเทอร์รัพขึ้นจะมีการเรียกใช้โปรแกรมย่อย crane_interrupt_interrupt และ ยังแสดงให้เห็นถึงวิธีการตรวจสอบหมายเลขอินเทอร์รัพที่ต้องการใช้งาน ว่าสามารถใช้งานได้หรือไม่โดยอินเทอร์รัพที่ใช้ในการทดลองนี้คืออินเทอร์รัพหมายเลข 7 ซึ่งเป็นอินเทอร์รัพของปรีนเตอร์

3.5.1.3 โปรแกรมส่วนควบคุมสแต็ปปีงมอเตอร์

โปรแกรมส่วนนี้เป็นส่วนควบคุมการทำงานของสแต็ปปีงมอเตอร์ทั้งหมด ตั้งแต่การรับข้อมูลตำแหน่งจากผู้ใช้และตรวจสอบความถูกต้องของข้อมูลที่รับเข้ามาไปจนถึงตรวจสอบที่หมายปลายทางที่เกนกำลังเคลื่อนที่ เป็นต้น

```

static int proc_move_only(struct file *file, const char *buffer,unsigned long count, void *data)

```

ตัวอย่างของโปรแกรมด้านบนแสดงให้เห็นโครงสร้างของการส่งค่าพารามิเตอร์ในการเรียกใช้งาน Process file ที่กำลังทำงานอยู่ในเคอร์เนล

```

{
    long i;
    int tempx,tempy;
    /*char xdirect,ydirect; */
    /* check data input correct */
    if(count > 3)
    {
        printk("user request position more than 2 digit\n");
    }
}

```

```

        get_status_file->data = user_position_error; //set status text to user mistake
        goto nothing_to_do;
    }
    else if(count < 3)
    {
        printk("user request position less than 2 digit\n");
        get_status_file->data = user_position_error; //set status text to user mistake
        goto nothing_to_do;
    }
    else
        printk("move to x=%c y=%c\n",buffer[0],buffer[1]);

    /*change char to int for compare*/
    temp_x = chartoint(buffer[0]);          /*set x target to temp_x */
    temp_y = chartoint(buffer[1]);          /*set y target to temp_y */

    /* check for valid number */
    if((temp_x== -1) || (temp_y== -1))
    {
        printk("user not keyin number\n");
        get_status_file->data = user_keyin_error;    //set status text to user mistake
    }

```

ส่วนของโปรแกรมด้านบนทำหน้าที่ตรวจสอบความถูกต้องของข้อมูลที่ผู้ใช้ป้อนให้กับระบบ ถ้าข้อมูลที่ป้อนไม่ถูกต้องเช่น ไม่ใช่ตัวเลข หรือป้อนเกินค่าที่กำหนดจะแสดงข้อความผิดพลาด

```

else    /* every thing ok so we move the motor */
{
    printk("x and y motor moving\n");
    get_status_file->data = moving_string ;
    if((temp_x==0)&&(temp_y==0))
        move_to_00();
    else
    {

```

```

timeout =0;
xyposchange = 0;
while(!((temp_x == xpos) && (temp_y == ypos))&&(timeout <
    MAXTIMEOUTDELAY))
    /* wait for x and y position */
{
    /* check x position for move */
    if(temp_x < xpos)
        xdirect = 'r';    /* move reverse direction */
    else if(temp_x > xpos)
        xdirect = 'f';    /* move forward direction */
    else
        xdirect = 'n';    /* not move */
    /* check y position for move */
    if(temp_y < ypos)
        ydirect = 'r';    /* move reverse direction */
    else if(temp_y > ypos)
        ydirect = 'f';    /* move forward direction */
    else
        ydirect = 'n';    /* not move */

    move_x_motor();
    move_y_motor();
    send_outport();
    mydelay(MAXMOTORDELAY);
    timeout ++;
}

printf("crane stand by \n");
get_status_file->data = standby_string ;
}
}

if(timeout >= MAXTIMEOUTDELAY)
{
    get_status_file->data = timeout_error_string ;
}

```

```

        move_to_00();
    }

```

เมื่อข้อมูลที่ป้อนเข้ามาถูกต้อง จากนั้นโปรแกรมจะทำการหมุนมอเตอร์ โดยจะมีจุดสิ้นสุดเมื่อค่าของตำแหน่งเครนเท่ากับตำแหน่งที่ต้องการ แต่ถ้าเครนไม่สามารถไปยังจุดที่กำหนดได้ทันเวลาจะมีการสั่งให้เครนเคลื่อนไปยังตำแหน่ง 0,0

3.5.1.4 โปรแกรมส่วนการทำงานเมื่อเกิดอินเทอร์รัพ

เป็นส่วนของโปรแกรมที่จะถูกเรียกใช้งานเมื่อเกิด หมายเลขอินเทอร์รัพตามที่ได้ลงทะเบียนไว้ ขณะทำเริ่มต้นระบบ โดยมีหน้าที่ในการควบคุมการตอบสนองต่อการเกิดอินเทอร์รัพของโปรแกรม

```

void crane_interrupt_interrupt(int irq, void *dev_id, struct pt_regs *regs)
{
    CheckInputAndAction();
}

```

ตัวอย่างของโปรแกรมด้านบนแสดงให้เห็น โครงสร้างการส่งค่าพารามิเตอร์ที่เกิดขึ้นขณะที่โปรเซสถูกเรียกผ่านอินเทอร์รัพโปรเซสที่เกิดขึ้นในเคอร์เนลและได้มีการเรียกใช้โปรแกรมย่อยอีกครั้งหนึ่ง

```

void CheckInputAndAction()
{
    int ipA;
    outb(SPPPORTREAD|INPUTENABLE,SPPCONTROLPORT);
    ipA = inb(SPPDATAPORT);
    if(ipA & 0x80)                /*bottom limit switch */
    {
        ydirect = 'n';
        ypos = 0;
    }
    if(ipA & 0x20)                /* top limit switch */
    {
        ydirect = 'n';
        ypos = 6;
    }
}

```

```

        if(ipA & 0x40)                /*left limit switch */
        {
            xdirect = 'n';
            xpos = 0;
        }
        if(ipA & 0x10)                /* right limit switch */
        {
            xdirect = 'n';
            xpos = 6;
        }
        if(ipA & 0x01)                /* x sensor */
        {
            /* x sensor action see details in appendix */
        }
        if(ipA & 0x02)                /* y sensor */
        {
            /* y sensor action see details in appendix */
        }
        outb(SPPINTERRUPTENABLE|OUTPUTENABLE|OUTPUTLATCH,SPPCONTROLPORT);
    }

```

โปรแกรมย่อยด้านบนจะถูกเรียกใช้งานเมื่อเกิดอินเตอร์รัพ โดยโปรแกรมจะอ่านค่าอินพุตจากพอร์ตนานเพื่อหาที่มาของสัญญาณอินเตอร์รัพ เมื่อหาจนพบจากนั้นโปรแกรมจะทำงานตามโปรแกรมย่อยที่กำหนดต่อไป

3.5.1.5 โปรแกรมส่วนแสดงข้อมูลให้กับผู้ใช้

ดังที่กล่าวไว้แล้วว่าข้อมูลที่โปรแกรมส่งให้กับผู้ใช้มีเพียงสถานะและตำแหน่ง ตัวอย่างซอร์สโค้ดด้านล่างเป็นโครงสร้างของโปรแกรมที่ทำหน้าที่ส่งค่ากลับไปให้กับผู้ใช้งาน

```
/*proc_read_position
```

```
* this process used for send crane position to user when user read position_file
```

```
*/
```

```
static int proc_read_position(char *page,char **start, off_t off, int count, int *eof, void *data)
```

```

{
    page[0] = (char) (xpos + 48);
    page[1] = (char) (ypos + 48);
    page[2] = '\n';
    return 3;
}

/*proc_read_status
 * this process used for send crane status to user when user read status_file
 */

static int proc_read_status(char *page,char **start, off_t off, int count, int *eof, void *data)
{
    char *outchar = (char *)data;
    int i=0;
    while((outchar[i] != '\n') && (i<255))
    {
        page[i] = outchar[i];
        i++;
    }
    page[i] = '\n';
    i++;
    return i;
}

```

3.5.2 โปรแกรมติดต่อกับผู้ใช้ผ่านเว็บเพจ

โปรแกรมนี้จะถูกเก็บไว้ในไดเรกทอรีของเว็บเซิร์ฟเวอร์ที่รองรับการทำงาน CGI Script โดยตำแหน่งไดเรกทอรีบน debian linux นั้นจะอยู่ที่ /usr/lib/cgi-bin/

```
#!/usr/bin/perl
```

```
#####
```

```
# crane web interface script\
```

```
#
```

```
# created by sthaporn
# 10/10/47      rev.2

#####

# read data from user request
read(STDIN, $dat, $ENV{'CONTENT_LENGTH'});
```

ด้านบนเป็นส่วนของโปรแกรมที่ทำหน้าที่รับค่าอินพุตที่ผู้ใช้งานส่งมายังสคริปต์

```
if(length($dat)==0)
{
    $xpos=" ";
    $ypos=" ";
}
else
{
    ($temp1,$temp2) = split(/&/,$dat);
    ($temp11,$temp12) = split(/=/,$temp1);
    ($xpos,$ypos) = split(/\%2C/,$temp12);
    `echo $xpos$ypos > /proc/crane_controller/move_only`;
}
```

จากนั้นโปรแกรมจะทำการตรวจสอบค่าอินพุตที่รับเข้ามาว่ามีหรือไม่

```
# set base color
$white="FFFFFF";
$red="FF0000";
@color = [];
# reset all cells color to white
$color[11]=$white;

#####

# reset all cell to white color

#####

$color[55]=$white;
```


จากนั้นโปรแกรมจะทำการสร้างอาร์ตเท่ากับจำนวนช่องของโครงและทำการตั้งค่าทุกช่องให้เป็นสีขาวทั้งหมด

```
#####
```

```
# read condition and set display here
```

```
#####
```

```
$status = `cat /proc/crane_controller/crane_status`;
```

```
($status_no,$status_text)=split(/\./,$status);
```

```
$xypos = `cat /proc/crane_controller/crane_position`;
```

```
$color[$xypos] = $red;
```

จากนั้น โปรแกรมจะทำการอ่านค่าตำแหน่งและสถานะจาก Process file ทั้ง crane_status และ crane_position จากนั้นจะนำค่าที่อ่านได้เก็บไว้ในตัวแปร \$xypos เพื่อใช้ชี้ตารางที่ต้องการระบายแดงในบรรทัดสุดท้าย

```
#####
```

```
# display html page script
```

```
#####
```

```
print "Content-type: text/html\n\n";
```

```
print "<html>\n";
```

```
if($status_no==5)
```

```
{
```

```
    print "<meta http-equiv='refresh' content='1;url=http://192.168.0.50/cgi-bin/main.cgi'>\n";
```

```
}
```

```
    print "<title>Crane controller project\n";
```

```
    print "</title>\n";
```

```
    print "<body>Rack Display\n";
```

```
    print "<table border='1' width='25%'>\n";
```

```
    print "<tr>\n";
```

```
    print "<td width='20%' align='center' bgcolor='\"$color[15]\">15</td>\n";
```

```
#####
```

```
# draw rack table
```

```
#####
```

```
    print "<td width='20%' align='center' bgcolor='\"$color[51]\">51</td>\n";
```

```

print "</tr>\n";

print "</table>\n";
print "<br>${status_text}\n";
print "<br><form method=\"POST\" action=\"./main.cgi\">\n";
print "<br>Please fill X and Y position in format X,Y<br>\n";
print "<br><input type=\"text\" name=\"xyposition\"><br>\n";
print "<br><input type=\"submit\" value=\"Start move\" name=\"moveb\">\n";
print "<input type=\"reset\" value=\"Clear\" name=\"resetb\">\n";
print "</form>\n";
print "</body>\n";
print "</html>\n";

```

ส่วนสุดท้ายของโปรแกรมเป็นการสร้างฟอร์มสำหรับรับค่าตำแหน่งเป้าหมายของเครนจากผู้ใช้ และทำการส่งค่าตำแหน่งนั้นให้กับตัวเองเพื่อให้เริ่มการทำงานอีกครั้งหนึ่ง

3.5.3 ทดสอบการทำงาน

จากการทดสอบการทำงานของโมดูล crane_controller.o ด้วยวิธีการจำลองป้อนค่าอินพุต โดยตรงจากผู้ใช้และป้อนค่าตำแหน่งของเครนด้วยตัวของผู้ใช้เองพบว่าสามารถทำงานได้เป็นอย่างดีโดยโปรแกรมสามารถสร้าง Process file ทั้งหมดได้สำเร็จ และสามารถทำการส่งและรับข้อมูลตอบโต้กับผู้ใช้งานได้เป็นปกติ

ในส่วนโปรแกรมติดต่อกับผู้ใช้ผ่านเว็บเพจ main.cgi ผู้ใช้สามารถเปิดเว็บเบราว์เซอร์โดยเรียกยูอาร์แอลที่ <http://192.168.0.50/cgi-bin/main.cgi> (ขึ้นอยู่กับไอพีแอดเดรสที่ผู้ใช้งานตั้งไว้) จาก Client เครื่องอื่นพบว่าสามารถเปิดใช้งาน และ ส่งค่าที่ผู้ใช้ป้อนไปยัง Process file ได้อย่างถูกต้อง

3.6 การจัดทำเครนอินเตอร์เฟส

3.6.1 การเตรียมอุปกรณ์

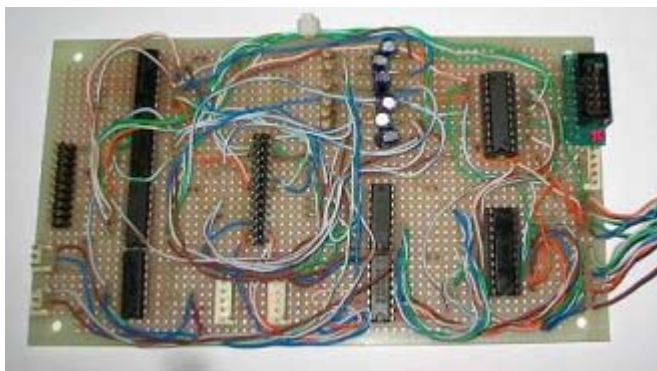
เตรียมอุปกรณ์อิเล็กทรอนิกส์ที่ใช้ในการทำงานดังนี้

แผ่นวงจรเอนกประสงค์	1	แผ่น
IC 74533	1	ตัว
IC 74244	1	ตัว
IC ULN2003	2	ตัว
IC 7408	2	ตัว
IC 7432	1	ตัว

IC 7404	1	ตัว
R 15K	6	ตัว
R 47K	6	ตัว
C 0.1uF	6	ตัว
male socket 25 pin	1	ตัว
male socket ตัวผู้ 2 pin	3	ตัว
parallel port female	1	ตัว
multimeter	1	ตัว

3.6.2 การประกอบวงจรทรานซิสเตอร์เพส

การประกอบวงจรจะทำการประกอบตามวงจรในรูปด้านล่าง ซึ่งวงจรที่ประกอบเสร็จแล้วมีลักษณะดังรูป



รูปที่ 3-18 แสดงแผ่นวงจรทรานซิสเตอร์เพสหลังจากประกอบอุปกรณ์เรียบร้อยแล้ว



รูปที่ 3-19 แสดงวงจรเพาเวอร์ซัพพลาย

3.6.3 ทดสอบการทำงาน

เมื่อประกอบแผ่นวงจรเรียบร้อยแล้วได้ทำการตรวจสอบการทำงานของแผ่นวงจรโดยการป้อนสัญญาณอินพุตเข้าไปยังวงจรโดยตรง ทั้งจาก dry contact และเซ็นเซอร์จากนั้นทำการวัดแรงดันที่เอาต์พุตด้วยมัลติมิเตอร์ผลปรากฏว่าแรงดันเอาต์พุตทุกจุดสามารถทำงานได้อย่างเป็นปกติ

เนื่องจากมัลติมิเตอร์มีความไวในการวัดต่ำจึงไม่สามารถวัดค่าความกว้างพัลส์ของสัญญาณอินพุตได้ ดังนั้นการตรวจสอบสัญญาณอินพุตจึงต้องทดสอบกับโปรแกรมใช้งานจริง

3.7 การจัดทำเครนและโครง

3.7.1 การเตรียมอุปกรณ์

อะลูมิเนียมหน้าตัดรูป T ขนาดหน้าตัด กว้าง 2.5cm. X สูง 2.5cm.	2	ท่อน
อะลูมิเนียมหน้าตัดรูป L ขนาดหน้าตัด กว้าง 2.0cm. X สูง 2.0cm.	1	ท่อน
ท่ออะลูมิเนียมสี่เหลี่ยม ขนาดหน้าตัด กว้าง 2.5cm. X ยาว 2.5cm.	1	ท่อน
อะลูมิเนียมเส้นแบน ขนาดหน้าตัด หน้า 2mm. X กว้าง 2cm.	1	เส้น
น็อตเกลียวปล่อยขนาด 3x10	1	กล่อง
น็อตตัวผู้และตัวเมียขนาด 3x10	1	กล่อง
สว่านไฟฟ้า	1	ตัว
เลื่อยเหล็ก	1	อัน

หมายเหตุ อะลูมิเนียมมาตรฐานยาว 6 เมตรต่อท่อน

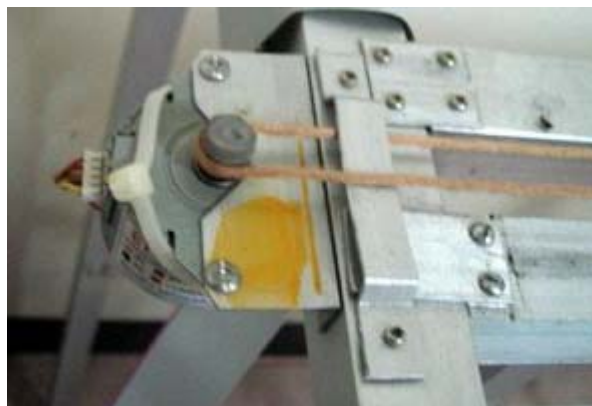
3.7.2 การประกอบอุปกรณ์

3.7.2.1 ส่วนโครงอะลูมิเนียม

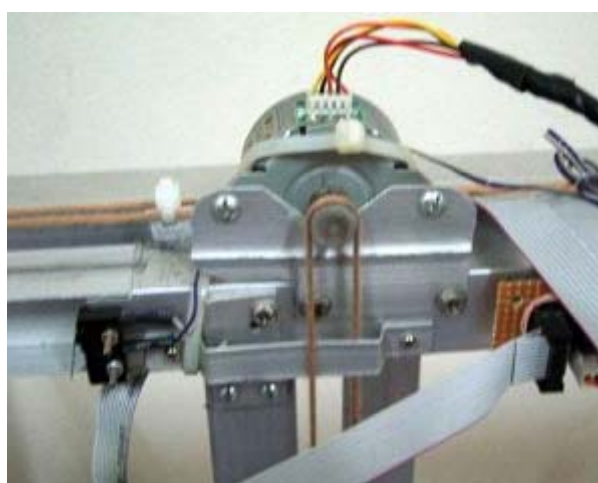
เป็นส่วนของการประกอบโครงอะลูมิเนียม ซึ่งประกอบด้วยการยึดติดแต่ละชิ้นส่วนของอะลูมิเนียมเข้าด้วยกันเพื่อให้ได้ตามที่ออกแบบไว้เพื่อสำหรับใช้ยึดแต่ละส่วนประกอบต่อไป

3.7.2.2 ส่วนติดตั้งสเต็ปมอเตอร์

การติดตั้งสเต็ปมอเตอร์เข้ากับเครนสามารถทำได้โดยทำแผ่นยึดมอเตอร์รูปตัวแอลเจาะรูแผ่นยึดเข้ากับโครงของโครงด้านหนึ่งและอีกด้านยึดติดกับมอเตอร์จากมอเตอร์ทั้ง 2 ตัวมีการใช้เคเบิลไทร์ (cable tie) ยึดด้านหลังไว้เพื่อช่วยต้านแรงดึงจากสายพานที่ทำให้มอเตอร์หมุนไม่คงที่



รูปที่ 3-20 แสดงการติดตั้งสแต็ปปีงมอเตอร์ควบคุมแนวนอน



รูปที่ 3-21 แสดงการติดตั้งสแต็ปปีงมอเตอร์ควบคุมแนวตั้ง

3.7.2.3 การติดตั้งลิมิตสวิตช์

ในการติดตั้งลิมิตสวิตช์จะกระทำเหมือนกันทุกจุดคือยึดติดกับส่วนปลายของส่วนเคลื่อนที่ทั้งแนวตั้งและแนวนอน จะมีเพียงลิมิตสวิตช์ด้านล่างเท่านั้นที่ไม่สามารถยึดติดกับเกรนได้เพราะพื้นที่ในการเจาะรูนั้นน้อยเกินไป



รูปที่ 3-22 แสดงการติดตั้งลิมิตสวิตช์เข้ากับส่วนเคลื่อนที่แนวนอน

3.7.2.4 โพล์รับสายพาน

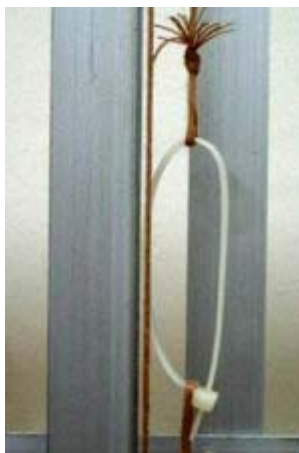
เนื่องจากระบบการหมุนของมอเตอร์ที่ผู้จัดทำออกแบบไว้ให้ใช้มอเตอร์เพียงตัวเดียวในการทำงาน ดังนั้นจึงต้องมีการสร้างโพล์ขึ้นมารองรับด้านตรงข้ามของมอเตอร์ดังรูป



รูปที่ 3-23 แสดงการติดตั้งโพล์ยึดสายพานแนวนอน

3.7.2.5 ส่วนปรับความตึงสายพาน

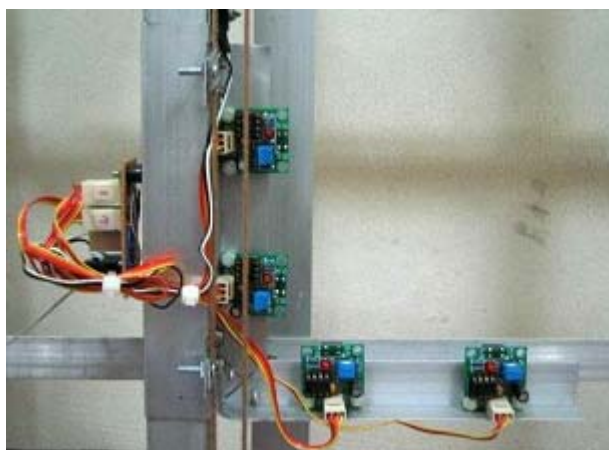
เนื่องจากสายพานที่นำมาใช้งานกับเครนได้ทำการประดิษฐ์สายพานขึ้นใช้เอง โดยสร้างจากเส้นด้ายที่เหนียวจำนวน 30 เส้นทำให้สายพานไม่มีความยืดหยุ่นในการทำงาน จึงต้องมีการใช้เคเบิลไทร์เพื่อใช้เป็นตัวปรับความตึงหย่อนของสายพานดังรูป



รูปที่ 3-24 แสดงการใช้เคเบิลไทรในการปรับความตึงของสายพาน

3.7.2.6 การติดตั้งอุปกรณ์เซ็นเซอร์

การติดตั้งอุปกรณ์เซ็นเซอร์เข้ากับเกรนไม่มีความซับซ้อนมากนัก จะมีจุดเดียวที่ต้องระวังคือ ระยะเซ็นเซอร์จะต้องไม่ห่างหรือใกล้กับโครงจนเกินไป เพราะอาจจะทำให้ไม่สามารถตรวจจับขอบของช่องไม่ได้ถ้าใกล้เกินไป หรืออาจจะทำให้ตัวเซ็นเซอร์ชูดกับโครงกรณีที่ติดตั้งใกล้เกินไป การติดตั้งเซ็นเซอร์แสดงดังรูป



รูปที่ 3-25 แสดงการติดตั้งเซ็นเซอร์สำหรับตรวจจับการเคลื่อนที่

3.7.2.7 ส่วนประกอบอื่นๆ

ส่วนอื่นๆนอกเหนือจากที่กล่าวมาข้างต้นที่ควรให้ความสนใจได้แก่ น็อตและสกรูต่างๆที่ยึดติดกับส่วนเคลื่อนที่ ซึ่งต้องคำนึงถึงความยาวของน็อตจะต้องไม่ให้ยาวจนเกินไป เพราะอาจจะไปสัมผัสกับรางหรือโครงได้ ซึ่งจะมีผลต่อการเคลื่อนที่ของเกรนอย่างมาก รูปที่สมบูรณ์ของอุปกรณ์ทั้งหมดเมื่อประกอบเข้าด้วยกันแล้วแสดงดังรูป



รูปที่ 3-26 แสดงโครงและโครงที่สมบูรณ์

3.7.3 ทดสอบการทำงาน

การทดสอบการทำงานของเครนสามารถแบ่งเป็นส่วนๆได้ดังนี้

- ทดสอบความมั่นคงแข็งแรงของโครงทำโดยกดบนโครงด้วยแรงพอประมาณ พบว่าสามารถรองรับน้ำหนักได้เป็นปกติ จากนั้นทำการทดสอบโดยการเอียงโครงว่าจะมีผลต่อการเคลื่อนที่ของเครนหรือไม่ พบว่าถ้าโครงเอียงมากจะทำให้เครนเคลื่อนที่ได้ยากไปด้วย
- ทดสอบการเคลื่อนที่ของส่วนเคลื่อนที่ทั้งแนวนอนและแนวตั้งด้วยมือ พบว่ามีความฝืดเล็กน้อยสามารถแก้ไข โดยการตะไบขอบของรางบางส่วนและเติมน้ำมันหล่อลื่นเล็กน้อย
- ทดสอบระยะในการเคลื่อนที่ทุกส่วน เช่น ความห่างระหว่างเครนกับโครง และความห่างของสายแพรกับส่วนเคลื่อนที่ พบว่าระยะห่างเหมาะสมและสามารถทำงานได้เป็นปกติ
- ทดสอบการส่งสัญญาณจากเซ็นเซอร์ทุกตัวมายังอินเตอร์เฟสการ์ดโดยการใช้มัลติมิเตอร์พบว่าสามารถทำงานได้เป็นปกติทุกส่วน